

Universidad Tecnológica Metropolitana

Tecnologías de la Información Innovación Digital

Bases de datos en la nube

PROYECTO INTEGRADOR DE UNIDAD I

CLAUDENT: Sistema seguro de gestión de
información odontológica en la nube

Maestro: I.S. Lizandro Israel Reyes Carrillo

Equipo 2 — Integrantes:

Canul Cua Jesús Guadalupe

Peña Herrera Bogarth Alejandro

Rodríguez Rangel Yael

Solis Baas Diego Alberto

7° Cuatrimestre — Grupo A

Fecha de entrega: viernes 26 de septiembre de 2025

Índice General

- 1. Introducción**
- 2. Planteamiento de la problemática y justificación**
- 3. Objetivos**
- 4. Diseño de la base de datos**
 - 4.1 Modelo Entidad-Relación
 - 4.2 Diccionario de datos
 - 4.3 Documentación de la normalización (1FN, 2FN, 3FN)
- 5. Implementación cloud y poblado**
- 6. Índices y optimización de rendimiento**
- 7. Consultas de manipulación y analítica**
- 8. Seguridad y administración de accesos**
- 9. Referencias bibliográficas**
- 10. Anexos**

1. Introducción

Las bases de datos relacionales constituyen el pilar fundamental para el almacenamiento estructurado de información en organizaciones de todos los tamaños. Con la evolución hacia la computación en la nube, los servicios Database as a Service (DBaaS) como Supabase permiten desplegar, administrar y escalar bases de datos PostgreSQL sin la complejidad de gestionar infraestructura física.

La seguridad de la información se ha convertido en un aspecto crítico en la administración de bases de datos. Un sistema mal configurado, donde todos los usuarios comparten credenciales de administrador, representa un riesgo significativo de fuga, modificación no autorizada o eliminación accidental de datos sensibles. PostgreSQL ofrece un robusto sistema de control de acceso basado en roles (RBAC) mediante sentencias DCL como CREATE ROLE, GRANT y REVOKE.

El presente proyecto integrador aborda el diseño, implementación y aseguramiento de una base de datos relacional para una clínica odontológica ficticia denominada CLAUDENT, aplicando normalización 3FN, índices, triggers de auditoría, consultas analíticas complejas y administración granular de privilegios.

2. Planteamiento de la problemática y justificación

Descripción del contexto real

Las clínicas odontológicas manejan información sensible de pacientes, tratamientos, citas, pagos y personal médico. Muchas clínicas pequeñas usan procesos manuales que provocan problemas de organización, pérdida de información y falta de control de accesos.

Problemática seleccionada

Falta de una plataforma centralizada y segura para administrar la información clínica y administrativa de una clínica odontológica, generando problemas de organización, seguridad y control de acceso a los datos.

Justificación técnica

El desarrollo de una BD relacional en la nube centralizará la información e implementará mecanismos de seguridad mediante roles, privilegios y auditoría. Supabase ofrece escalabilidad, alta disponibilidad y una interfaz moderna sin inversión en infraestructura.

3. Objetivos

Objetivo general

Diseñar e implementar una base de datos relacional en la nube para una clínica odontológica que permita administrar información clínica y administrativa aplicando mecanismos de seguridad y control de acceso.

Objetivos específicos

- Técnico: Arquitectura relacional normalizada en 3FN con restricciones, índices y triggers.
- Funcional: 8 consultas SQL analíticas complejas con JOINS y funciones de agregación.
- De seguridad: 3 roles con privilegios diferenciados (DCL) y 5 pruebas de acceso.

Alcance

- Registro y administración de pacientes.
- Control de odontólogos y especialidades.
- Gestión de citas y consultas.
- Registro de tratamientos y pagos.
- Administración de usuarios y permisos.
- Auditoría automática mediante trigger.

Requerimientos funcionales

- Registrar pacientes con información personal única.
- Gestionar citas con estados controlados (PROGRAMADA, ATENDIDA, CANCELADA).
- Registrar consultas/tratamientos con diagnóstico.
- Controlar pagos por múltiples métodos.
- Bitácora de auditoría automática con trigger.

4. Diseño de la base de datos

El esquema consta de 10 tablas normalizadas en 3FN dentro del esquema claudent en PostgreSQL (Supabase).

4.1 Modelo Entidad-Relación

[INSERTAR: Captura del Diagrama ER (dbdiagram.io / Draw.io)]

Figura 1. Diagrama Entidad-Relación del esquema CLAUDENT en 3FN.

Relaciones principales:

- rol 1:N usuario
- especialidad 1:N dentista
- paciente 1:N cita
- dentista 1:N cita
- cita 1:1 consulta
- servicio 1:N consulta
- consulta 1:N pago
- usuario 1:N bitacora_auditoria

4.2 Diccionario de datos

Tabla: rol

Campo	Tipo	Restricciones
id_rol	SERIAL	PK
nombre_rol	VARCHAR(50)	UNIQUE, NOT NULL
descripcion	VARCHAR(200)	—

Tabla: usuario

Campo	Tipo	Restricciones
id_usuario	SERIAL	PK
id_rol	INTEGER	FK→rol
nombre_usuario	VARCHAR(80)	NOT NULL
correo	VARCHAR(120)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL
estado	VARCHAR(20)	CHECK

Tabla: especialidad

Campo	Tipo	Restricciones
id_especialidad	SERIAL	PK
nombre_especialidad	VARCHAR(100)	UNIQUE,NOT NULL
descripcion	VARCHAR(200)	—

Tabla: dentista

Campo	Tipo	Restricciones
id_dentista	SERIAL	PK
id_especialidad	INTEGER	FK→especialidad
nombre	VARCHAR(80)	NOT NULL
apellido	VARCHAR(80)	NOT NULL
telefono	VARCHAR(15)	UNIQUE
correo	VARCHAR(120)	UNIQUE
cedula_profesional	VARCHAR(50)	UNIQUE

Tabla: paciente

Campo	Tipo	Restricciones
id_paciente	SERIAL	PK
nombre	VARCHAR(80)	NOT NULL
apellido	VARCHAR(80)	NOT NULL
fecha_nacimiento	DATE	—
telefono	VARCHAR(15)	UNIQUE
correo	VARCHAR(120)	UNIQUE
direccion	VARCHAR(200)	—
fecha_registro	DATE	DEFAULT

Tabla: servicio

Campo	Tipo	Restricciones
id_servicio	SERIAL	PK
nombre_servicio	VARCHAR(100)	UNIQUE,NOT NULL
descripcion	VARCHAR(250)	—
costo	NUMERIC(10,2)	CHECK>0
estado	VARCHAR(20)	CHECK

Tabla: cita

Campo	Tipo	Restricciones
id_cita	SERIAL	PK
id_paciente	INTEGER	FK→paciente
id_dentista	INTEGER	FK→dentista
fecha_cita	DATE	NOT NULL
hora_cita	TIME	NOT NULL
motivo	VARCHAR(200)	—
estado	VARCHAR(20)	CHECK

Tabla: consulta

Campo	Tipo	Restricciones
id_consulta	SERIAL	PK
id_cita	INTEGER	FK→cita
id_servicio	INTEGER	FK→servicio
diagnostico	VARCHAR(300)	—
observaciones	VARCHAR(300)	—
fecha_consulta	DATE	DEFAULT

Tabla: pago

Campo	Tipo	Restricciones
id_pago	SERIAL	PK
id_consulta	INTEGER	FK→consulta
monto	NUMERIC(10,2)	CHECK>0
metodo_pago	VARCHAR(30)	CHECK
fecha_pago	DATE	DEFAULT

Tabla: bitacora_auditoria

Campo	Tipo	Restricciones
id_bitacora	SERIAL	PK
id_usuario	INTEGER	FK→usuario
operacion	VARCHAR(20)	NOT NULL
tabla_afectada	VARCHAR(50)	NOT NULL
fecha_evento	TIMESTAMP	DEFAULT
descripcion	TEXT	—

4.3 Documentación del proceso de normalización

Primera Forma Normal (1FN):

- Todos los campos contienen valores atómicos.
- No existen grupos repetitivos ni arrays.
- Nombres separados en nombre y apellido (no nombre_completo).

Segunda Forma Normal (2FN):

- Cumple 1FN + cada atributo depende completamente de la PK.
- PKs simples (SERIAL), sin dependencias parciales.

Tercera Forma Normal (3FN):

- Cumple 2FN + no hay dependencias transitivas.
- Catálogos independientes: rol, especialidad y servicio en tablas separadas con FK.
- consulta separa la relación cita-servicio.
- Estados con CHECK (dominios fijos propios de cada entidad).

Resultado: Las 10 tablas cumplen 1FN, 2FN y 3FN.

5. Implementación cloud y poblado

Orden de creación DDL (sin dependencias circulares):

- 1. rol
- 2. usuario
- 3. especialidad
- 4. dentista
- 5. paciente
- 6. servicio
- 7. cita
- 8. consulta
- 9. pago
- 10. bitacora_auditoria

Metodología de poblamiento con IA:

"Genera registros ficticios realistas para una clínica odontológica en Mérida, Yucatán. Nombres mayas/mestizos, costos reales, citas en sept. 2025, respetando UNIQUE, CHECK y FKs."

Tabla	Registros
rol	3
usuario	4
especialidad	5
dentista	5
paciente	10
servicio	8
cita	12
consulta	8
pago	8
bitacora	3+auto
TOTAL	63+

[INSERTAR: Captura del Table Editor de Supabase con tablas activas]

Figura 2. Tablas del esquema claudent en Supabase.

[INSERTAR: Captura del visualizador de relaciones de Supabase]

Figura 3. Relaciones del esquema en Supabase.

6. Índices y optimización de rendimiento

Se crearon 14 índices estratégicos basados en los patrones de consulta operativos.

Índice	Columna(s)	Justificación
idx_cita_fecha	cita(fecha_cita)	Agenda diaria
idx_cita_estado	cita(estado)	Filtrar por estado
idx_cita_paciente	cita(id_paciente)	Historial paciente
idx_cita_dentista	cita(id_dentista)	Agenda dentista
idx_pac_apellido	paciente(apellido)	Búsqueda recepción
idx_pac_nom_ape	paciente(nom,ape)	Búsqueda combinada
idx_consulta_fecha	consulta(fecha)	Reportes diarios
idx_pago_fecha	pago(fecha_pago)	Cortes de caja
idx_pago_metodo	pago(metodo_pago)	Reportes financieros
idx_bit_fecha	bitacora(fecha)	Auditoría temporal
idx_bit_tabla	bitacora(tabla)	Auditoría por entidad
idx_dent_espec	dentista(id_espec)	Filtrar especialidad
idx_usr_rol	usuario(id_rol)	Filtrar por rol

Trigger de auditoría automática

Trigger que registra INSERT/UPDATE/DELETE en paciente automáticamente.

```
CREATE OR REPLACE FUNCTION fn_auditoria_paciente()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO claudent.bitacora_auditoria
        (operacion, tabla_afectada, descripcion)
    VALUES (TG_OP, 'paciente',
        CASE
            WHEN TG_OP = 'INSERT' THEN 'Nuevo: ' || NEW.nombre
            WHEN TG_OP = 'UPDATE' THEN 'Actualizado ID: ' || OLD.id_paciente
            WHEN TG_OP = 'DELETE' THEN 'Eliminado: ' || OLD.nombre
        END);
    RETURN COALESCE(NEW, OLD);
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auditoria_paciente
AFTER INSERT OR UPDATE OR DELETE ON paciente
FOR EACH ROW EXECUTE FUNCTION fn_auditoria_paciente();
```

7. Consultas de manipulación y analítica

8 consultas SQL complejas con INNER JOIN, LEFT JOIN, COUNT, SUM, AVG, GROUP BY, HAVING, COALESCE y ORDER BY.

C1: Pacientes con citas programadas

INNER JOIN paciente-cita-dentista, WHERE estado='PROGRAMADA'.

```
SELECT p.nombre||' '||p.apellido AS paciente,
       d.nombre||' '||d.apellido AS dentista,
       c.fecha_cita, c.hora_cita, c.motivo
FROM cita c
INNER JOIN paciente p ON c.id_paciente=p.id_paciente
INNER JOIN dentista d ON c.id_dentista=d.id_dentista
WHERE c.estado='PROGRAMADA'
ORDER BY c.fecha_cita;
```

[INSERTAR: Captura de resultados Consulta 1]

Figura 4. Resultados de la consulta 1.

C2: Pacientes atendidos por odontologo

COUNT(DISTINCT)+GROUP BY+LEFT JOIN.

```
SELECT d.nombre||' '||d.apellido AS dentista,
       e.nombre_especialidad,
       COUNT(DISTINCT c.id_paciente) AS pacientes
FROM dentista d
INNER JOIN especialidad e ON d.id_especialidad=e.id_especialidad
LEFT JOIN cita c ON d.id_dentista=c.id_dentista
       AND c.estado='ATENDIDA'
GROUP BY d.id_dentista,d.nombre,d.apellido,e.nombre_especialidad
ORDER BY pacientes DESC;
```

[INSERTAR: Captura de resultados Consulta 2]

Figura 5. Resultados de la consulta 2.

C3: Ingresos por servicio

SUM+AVG+COUNT cruzando servicio-consulta-pago.

```
SELECT s.nombre_servicio,  
       COUNT(co.id_consulta) AS veces,  
       SUM(pa.monto) AS ingreso_total,  
       AVG(pa.monto) AS promedio  
FROM servicio s  
INNER JOIN consulta co ON s.id_servicio=co.id_servicio  
INNER JOIN pago pa ON co.id_consulta=pa.id_consulta  
GROUP BY s.id_servicio,s.nombre_servicio  
ORDER BY ingreso_total DESC;
```

[INSERTAR: Captura de resultados Consulta 3]

Figura 6. Resultados de la consulta 3.

C4: Analisis por metodo de pago

AVG+SUM+COUNT+MIN+MAX+GROUP BY.

```
SELECT metodo_pago, COUNT(*) AS cantidad,  
       SUM(monto) AS total, AVG(monto) AS promedio,  
       MIN(monto) AS minimo, MAX(monto) AS maximo  
FROM pago  
GROUP BY metodo_pago  
ORDER BY total DESC;
```

[INSERTAR: Captura de resultados Consulta 4]

Figura 7. Resultados de la consulta 4.

C5: Tratamientos mas solicitados (HAVING)

COUNT+GROUP BY+HAVING filtra populares.

```
SELECT s.nombre_servicio, s.costo,  
       COUNT(co.id_consulta) AS veces,  
       SUM(pa.monto) AS ingresos  
FROM servicio s  
INNER JOIN consulta co ON s.id_servicio=co.id_servicio  
INNER JOIN pago pa ON co.id_consulta=pa.id_consulta  
GROUP BY s.id_servicio,s.nombre_servicio,s.costo  
HAVING COUNT(co.id_consulta)>=1  
ORDER BY veces DESC;
```

[INSERTAR: Captura de resultados Consulta 5]

Figura 8. Resultados de la consulta 5.

C6: Pacientes sin citas (LEFT JOIN+IS NULL)

Detecta pacientes inactivos.

```
SELECT p.id_paciente,  
       p.nombre||' '||p.apellido AS paciente,  
       p.telefono, p.correo  
FROM paciente p  
LEFT JOIN cita c ON p.id_paciente=c.id_paciente  
WHERE c.id_cita IS NULL;
```

[INSERTAR: Captura de resultados Consulta 6]

Figura 9. Resultados de la consulta 6.

C7: Historial completo (6 tablas)

INNER JOIN paciente-cita-consulta-dentista-servicio-pago.

```
SELECT p.nombre||' '||p.apellido AS paciente,
       ci.fecha_cita, d.nombre||' '||d.apellido AS dentista,
       s.nombre_servicio, co.diagnostico,
       pa.monto, pa.metodo_pago
FROM paciente p
INNER JOIN cita ci ON p.id_paciente=ci.id_paciente
INNER JOIN consulta co ON ci.id_cita=co.id_cita
INNER JOIN dentista d ON ci.id_dentista=d.id_dentista
INNER JOIN servicio s ON co.id_servicio=s.id_servicio
INNER JOIN pago pa ON co.id_consulta=pa.id_consulta
ORDER BY p.apellido,ci.fecha_cita;
```

[INSERTAR: Captura de resultados Consulta 7]

Figura 10. Resultados de la consulta 7.

C8: Reporte mensual por dentista

SUM+COUNT DISTINCT+COALESCE+LEFT JOIN+GROUP BY.

```
SELECT d.nombre||' '||d.apellido AS dentista,
       e.nombre_especialidad,
       COUNT(DISTINCT ci.id_cita) AS citas,
       COALESCE(SUM(pa.monto),0) AS ingresos
FROM dentista d
INNER JOIN especialidad e ON d.id_especialidad=e.id_especialidad
LEFT JOIN cita ci ON d.id_dentista=ci.id_dentista
      AND ci.estado='ATENDIDA'
LEFT JOIN consulta co ON ci.id_cita=co.id_cita
LEFT JOIN pago pa ON co.id_consulta=pa.id_consulta
GROUP BY d.id_dentista,d.nombre,d.apellido,e.nombre_especialidad
ORDER BY ingresos DESC;
```

[INSERTAR: Captura de resultados Consulta 8]

Figura 11. Resultados de la consulta 8.

8. Seguridad y administración de accesos

8.1 Matriz de roles y privilegios

Tabla	Admin	Odontólogo	Recepcionista
rol	ALL	—	—
usuario	ALL	—	—
especialidad	ALL	SELECT	SELECT
dentista	ALL	SELECT	SELECT
paciente	ALL	SELECT	SEL,INS,UPD
servicio	ALL	SELECT	SELECT
cita	ALL	SEL,UPD	SEL,INS,UPD
consulta	ALL	SEL,INS,UPD	—
pago	ALL	—	—
bitacora	ALL	—	—

8.2 Script DCL

```

CREATE ROLE claudent_admin WITH LOGIN PASSWORD 'Admin2025$Clau';
CREATE ROLE claudent_odontologo WITH LOGIN PASSWORD 'Odont2025$Clau';
CREATE ROLE claudent_recepcionista WITH LOGIN PASSWORD 'Recep2025$Clau';

GRANT USAGE ON SCHEMA claudent TO claudent_admin,
    claudent_odontologo, claudent_recepcionista;

GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA claudent TO claudent_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA claudent TO claudent_admin;

GRANT SELECT ON claudent.paciente,claudent.cita,claudent.consulta,
    claudent.servicio,claudent.especialidad,claudent.dentista
    TO claudent_odontologo;
GRANT INSERT,UPDATE ON claudent.consulta TO claudent_odontologo;
GRANT UPDATE(estado) ON claudent.cita TO claudent_odontologo;

GRANT SELECT ON claudent.paciente,claudent.cita,claudent.servicio,
    claudent.dentista,claudent.especialidad TO claudent_recepcionista;
GRANT INSERT,UPDATE ON claudent.paciente TO claudent_recepcionista;
GRANT INSERT,UPDATE ON claudent.cita TO claudent_recepcionista;

REVOKE ALL ON claudent.usuario FROM claudent_odontologo,claudent_recepcionista;
REVOKE ALL ON claudent.pago FROM claudent_recepcionista;

```

8.3 Evidencias de pruebas de seguridad

P1: Admin consulta todas las tablas

Resultado: PERMITIDO

```
SET ROLE claudent_admin;  
SELECT * FROM claudent.usuario LIMIT 3;  
SELECT * FROM claudent.pago LIMIT 3;  
-- Resultado: exitoso  
RESET ROLE;
```

[INSERTAR: Captura de consola SQL - P1: Admin consulta todas las tablas]

Evidencia de p1: admin consulta todas las tablas.

P2: Odontologo consulta pacientes

Resultado: PERMITIDO

```
SET ROLE claudent_odontologo;  
SELECT * FROM claudent.paciente LIMIT 3;  
-- Resultado: datos visibles  
RESET ROLE;
```

[INSERTAR: Captura de consola SQL - P2: Odontologo consulta pacientes]

Evidencia de p2: odontologo consulta pacientes.

P3: Recepcionista crea cita

Resultado: PERMITIDO

```
SET ROLE claudent_recepcionista;  
INSERT INTO claudent.cita(id_paciente,id_dentista,  
    fecha_cita,hora_cita,motivo)  
VALUES (2,1,'2025-10-01','09:00','Limpieza');  
-- Resultado: INSERT 0 1  
RESET ROLE;
```

[INSERTAR: Captura de consola SQL - P3: Recepcionista crea cita]

Evidencia de p3: recepcionista crea cita.

P4: Recepcionista intenta ver pagos

Resultado: BLOQUEADO

```
SET ROLE claudent_recepcionista;  
SELECT * FROM claudent.pago;  
-- ERROR: permission denied for table pago  
RESET ROLE;
```

[INSERTAR: Captura de consola SQL - P4: Recepcionista intenta ver pagos]

Evidencia de p4: recepcionista intenta ver pagos.

P5: Odontologo intenta crear usuarios

Resultado: BLOQUEADO

```
SET ROLE claudent_odontologo;  
INSERT INTO claudent.usuario(id_rol,nombre_usuario,  
    correo,password_hash)  
VALUES(2,'Intruso','x@mail.com','hash');  
-- ERROR: permission denied for table usuario  
RESET ROLE;
```

[INSERTAR: Captura de consola SQL - P5: Odontologo intenta crear usuarios]

Evidencia de p5: odontologo intenta crear usuarios.

9. Referencias bibliográficas

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation: GRANT. <https://www.postgresql.org/docs/16/sql-grant.html>

Supabase. (2024). Supabase Documentation: Database. <https://supabase.com/docs/guides/database>

Coronel, C., & Morris, S. (2019). Database Systems: Design, Implementation, and Management (13th ed.). Cengage Learning.

Date, C. J. (2019). An Introduction to Database Systems (8th ed.). Pearson.

10. Anexos

Anexo A: Enlace al proyecto cloud

[INSERTAR: URL del proyecto en Supabase]

Anexo B: Script SQL completo

Archivo adjunto: Script_Modelo_Relacional_Claudent.sql con DDL, índices, trigger, DML (63+ registros), 8 consultas, DCL y pruebas de seguridad.